

INSTITUTO FEDERAL GOIANO - CAMPUS URUTAÍ
SISTEMAS DE INFORMAÇÃO
DISCIPLINA: SISTEMAS DISTRIBUÍDOS
PROFESSOR: JÚNIO CÉSAR DE LIMA

FILAS DE MENSAGENS EM SISTEMAS DISTRIBUÍDOS

Comunicação indireta, desacoplamento e middleware orientado a mensagens

Aluno(a): Otávio de Queiroz Franco

Urutaí - GO
2026

FILAS DE MENSAGENS EM SISTEMAS DISTRIBUÍDOS

Resumo

As filas de mensagens constituem um dos principais mecanismos de comunicação indireta em sistemas distribuídos. Elas introduzem um intermediário entre produtores e consumidores, permitindo que processos troquem informações sem depender de conhecimento direto entre si e, em muitos casos, sem precisarem estar ativos ao mesmo tempo. Este relatório analisa o papel das filas de mensagens no contexto de middleware, destacando suas propriedades de desacoplamento espacial e temporal, seu modelo de programação, seus mecanismos de confiabilidade e suas principais formas de implementação. A discussão toma como base os materiais da disciplina de Sistemas Distribuídos, especialmente a aula sobre comunicação indireta e filas de mensagens, e complementa a análise com documentação oficial de IBM MQ, RabbitMQ, Spring AMQP e Jakarta Messaging. Conclui-se que filas de mensagens são especialmente relevantes para integração entre aplicações, processamento assíncrono, tolerância a falhas e construção de arquiteturas mais evolutíveis, embora introduzam custos de desempenho, operação e observabilidade.

Palavras-chave: Sistemas distribuídos. Comunicação indireta. Filas de mensagens. Middleware. RabbitMQ.

1 Introdução

Sistemas distribuídos são formados por componentes localizados em computadores conectados em rede, que coordenam suas ações por meio da troca de mensagens. Essa característica torna a comunicação um elemento central da disciplina: mais do que uma questão de transporte de bytes, comunicar em sistemas distribuídos envolve lidar com heterogeneidade, falhas parciais, latência, concorrência e escalabilidade. Os materiais introdutórios da disciplina destacam que redes de computadores e microprocessadores mais acessíveis permitiram a proliferação de sistemas compostos por vários nós, nos quais a passagem de mensagens é o mecanismo básico de interação entre componentes .

Nesse contexto, o middleware surge como uma camada de software posicionada entre as aplicações e a infraestrutura de rede e sistema operacional. Sua função é oferecer abstrações que reduzam a complexidade da distribuição, tais como chamadas remotas, serviços de nomes, transações, segurança e modelos de comunicação de mais alto nível . Entre esses modelos, as filas de mensagens se destacam por oferecerem uma forma de comunicação indireta, isto é, uma forma de interação na qual remetentes e destinatários não se comunicam diretamente, mas por meio de um intermediário.

O objetivo deste relatório é explicar o funcionamento das filas de mensagens e sua

importância em sistemas distribuídos. Para isso, o texto apresenta o conceito de comunicação indireta, descreve o modelo produtor-fila-consumidor, analisa propriedades como ordenação, persistência, validade, integridade e transações, e examina implementações como IBM MQ, Jakarta Messaging, RabbitMQ e Spring AMQP. Também são discutidas vantagens e limitações desse paradigma, considerando tanto sua utilidade acadêmica quanto sua aplicação em integração corporativa e arquiteturas modernas.

2 Comunicação indireta e desacoplamento

A comunicação indireta e caracterizada pela presença de um intermediário que separa os participantes da comunicação. Na comunicação direta, um processo precisa conhecer o endereco, a porta, o nome ou a identidade lógica do outro processo com o qual deseja interagir. Ja na comunicação indireta, a mensagem e depositada em uma entidade intermediária, como uma fila, um topico, um grupo ou um espaco compartilhado, e o receptor obtem a informação a partir dessa entidade .

Dois efeitos são particularmente relevantes: o desacoplamento espacial e o desacoplamento temporal. O desacoplamento espacial significa que o produtor não precisa conhecer diretamente quem consumirá a mensagem. Da mesma forma, o consumidor não precisa conhecer todos os produtores que podem abastecer a fila. Esse aspecto facilita substituição, replicação, migração e evolução de componentes, pois a dependência principal passa a ser o contrato da mensagem e o nome do destino intermediário.

O desacoplamento temporal, por sua vez, indica que produtor e consumidor podem ter ciclos de vida independentes. O produtor pode enviar uma mensagem mesmo que o consumidor esteja momentaneamente indisponível, desde que a infraestrutura de mensageria aceite e armazene a mensagem. Posteriormente, quando o consumidor voltar a operar, a mensagem poderá ser entregue. Essa propriedade é especialmente importante em ambientes sujeitos a falhas, oscilações de rede, manutenção de serviços e variações de carga.

É importante distinguir comunicação assíncrona de desacoplamento temporal. Uma chamada assíncrona permite que o emissor continue sua execução sem bloquear enquanto aguarda uma resposta. Entretanto, isso não garante, por si só, que o receptor possa estar ausente no momento da comunicação. Filas de mensagens frequentemente combinam os dois conceitos: o envio costuma ser assíncrono do ponto de vista da aplicação e a mensagem pode permanecer armazenada até que algum consumidor esteja disponível .

3 Modelo de funcionamento das filas de mensagens

Uma fila de mensagens é uma estrutura de comunicação na qual produtores inserem mensagens e consumidores as removem para processamento. O modelo básico é ponto a

ponto: uma mensagem colocada na fila é entregue a um único consumidor, ainda que vários consumidores possam estar associados a mesma fila. Essa característica diferencia filas de paradigmas como publicação/assinatura, nos quais um evento pode ser distribuído para múltiplos assinantes .

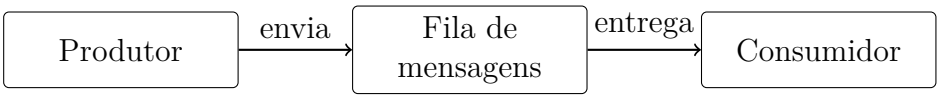


Figura 1: Modelo conceitual de comunicação por fila de mensagens.

O modelo de programação costuma ser simples. Um produtor envia uma mensagem para uma fila identificada por nome ou por algum destino lógico. Um consumidor, por sua vez, pode receber mensagens por diferentes estilos: recepção bloqueante, quando aguarda até que uma mensagem esteja disponível; recepção não bloqueante, quando consulta a fila e prossegue caso não haja mensagem; ou notificação, quando o middleware aciona o consumidor quando uma nova mensagem chega .

As mensagens normalmente possuem três partes principais. A primeira é o destino, que identifica a fila ou a entidade de roteamento. A segunda é o conjunto de metadados, como prioridade, modo de entrega, identificadores e propriedades. A terceira é o corpo da mensagem, que contém os dados da aplicação. Em muitos sistemas de mensageria, o corpo é tratado como opaco pelo broker, cabendo a aplicação definir se usará texto, bytes, JSON, XML, serialização binária ou outro formato. A documentação do IBM MQ, por exemplo, descreve a mensagem como informação significativa para as aplicações e separa dados de aplicação, descritor e propriedades .

4 Propriedades técnicas e garantias

Uma propriedade clássica das filas é a ordenação FIFO, isto é, a tendência de entregar primeiro a mensagem que entrou primeiro. Entretanto, essa ordenação pode ser modificada por recursos como prioridade, múltiplos consumidores, reentrega e particionamento. A documentação do RabbitMQ define filas como coleções ordenadas de mensagens, mas observa que prioridades e reentregas podem afetar a ordem observada pelos consumidores . Assim, em sistemas reais, FIFO deve ser entendido como uma semântica desejada e configurável, não como uma garantia absoluta em qualquer topologia.

Outra propriedade essencial é a persistência. Em uma fila persistente, mensagens podem ser armazenadas em disco para sobreviver a reinicializações ou indisponibilidades do broker. A aula de filas de mensagens enfatiza que essa persistência e central para entrega confiavel, pois permite que a mensagem permaneca na infraestrutura até ser consumida . No RabbitMQ, essa confiabilidade exige a combinação correta entre filas duraveis e mensagens publicadas como persistentes; se apenas um desses elementos for

configurado, a recuperacao apos falha pode não preservar todas as mensagens .

Do ponto de vista de corretude, duas propriedades aparecem com frequência: validade e integridade. A validade indica que uma mensagem enviada deve ser recebida, dentro das condições assumidas pelo sistema. A integridade indica que a mensagem recebida deve corresponder a mensagem enviada e que não deve haver duplicação indevida. Na prática, essas garantias dependem do broker, da confirmação de publicação, das confirmações de consumo, da persistência, das transações e da política de reentrega.

Transações ampliam essas garantias ao agrupar operações de envio ou recebimento dentro de uma unidade atomica. Em integrações corporativas, pode ser necessário que uma atualização em banco de dados e o envio de uma mensagem ocorram como uma unica decisão lógica: ou ambas acontecem, ou nenhuma deve produzir efeito permanente. Jakarta Messaging reconhece suporte a transações locais e distribuidas, incluindo integração com mecanismos XA em ambientes de aplicação . Esse tipo de recurso e valioso, mas também aumenta complexidade e custo operacional.

Por fim, filas de mensagens podem incorporar segurança, transformação e seleção. Sistemas comerciais oferecem autenticação, autorização e comunicação confidencial. A transformação de mensagens auxilia na interoperabilidade entre formatos heterogêneos, enquanto a seleção por propriedades permite que consumidores filtrem mensagens segundo critérios de aplicação. Esses recursos reforçam o papel das filas como middleware, e não apenas como estruturas de dados remotas.

5 Implementações e tecnologias relacionadas

O IBM MQ representa uma família tradicional de middleware orientado a mensagens, historicamente usada em integração de aplicações empresariais. Nesse modelo, gerenciadores de filas hospedam e administram filas, enquanto aplicações usam interfaces de programação para conectar, enviar, receber e consultar mensagens. A abordagem é adequada a ambientes nos quais confiabilidade, controle transacional, administração centralizada e integração entre plataformas distintas são requisitos relevantes .

Jakarta Messaging, sucessor do Java Message Service no ecossistema Jakarta EE, define uma API padronizada para aplicações Java que usam mensageria. A especificação diferencia clientes, mensagens, provedores e objetos administrados, além de reconhecer dois estilos principais: ponto a ponto, baseado em filas, e publicação/assinatura, baseado em tópicos . Sua importância está na portabilidade: uma aplicação pode programar contra uma API comum enquanto diferentes provedores implementam o transporte e as garantias concretas.

RabbitMQ é uma implementação amplamente utilizada de broker de mensagens, especialmente associada ao protocolo AMQP. Nele, produtores normalmente publicam mensagens em exchanges, e essas exchanges roteiam as mensagens para filas com base

em bindings e chaves de roteamento. A documentação oficial descreve exchanges como entidades que recebem publicações e encaminham mensagens para filas, streams ou outras exchanges; tipos como direct, fanout, topic e headers oferecem diferentes estratégias de roteamento . Esse modelo amplia a flexibilidade da fila simples, pois separa o ato de publicar do destino físico final.

Spring AMQP, por sua vez, fornece abstrações para aplicações Spring que interagem com RabbitMQ. O `RabbitTemplate` simplifica o envio de mensagens, enquanto anotações como `@RabbitListener` permitem declarar métodos consumidores associados a filas. Essa abordagem aparece também na aula prática, na qual um produtor envia uma mensagem para uma fila configurada por propriedade e um consumidor a recebe automaticamente . O resultado e um exemplo claro de comunicação indireta: produtor e consumidor dependem do contrato de mensagem e da fila, mas não de chamadas diretas entre si.

Tabela 1: Comparação entre modelos de comunicação

Critério	**Fila de mensagens**	**Publicação/ assinatura**	**Comunicação direta**
Relação entre partes	Um produtor envia para uma fila e um consumidor processa cada mensagem.	Um produtor publica eventos que podem chegar a vários assinantes.	Um processo envia diretamente a outro processo conhecido.
Acoplamento espacial	Baixo, pois o produtor conhece a fila ou rota, não o consumidor concreto.	Baixo, pois publicadores e assinantes são mediados por tópicos ou eventos.	Maior, pois o emissor depende do destino direto.
Acoplamento temporal	Pode ser baixo quando há persistência e consumidores podem processar depois.	Varia conforme retenção, durabilidade de assinaturas e broker.	Geralmente maior, sobretudo em chamadas sincronas.
Uso típico	Integração, processamento assíncrono, balanceamento de trabalho.	Disseminação de eventos, notificações, arquiteturas orientadas a eventos.	Requisição-resposta, chamadas de serviço, protocolos ponto a ponto.

6 Vantagens, limitações e cuidados de projeto

A principal vantagem das filas de mensagens é o desacoplamento. Ao introduzir uma fila entre produtor e consumidor, torna-se possível evoluir componentes de forma independente, absorver picos de carga, suavizar indisponibilidades temporarias e integrar sistemas implementados em tecnologias distintas. Em sistemas empresariais, essa característica favorece integração entre aplicações legadas, serviços modernos, rotinas batch e processos transacionais.

Outra vantagem e a escalabilidade operacional. Quando várias instâncias consumidoras leem de uma mesma fila, o trabalho pode ser distribuído entre elas. Esse padrão e util para tarefas demoradas, como processamento de pedidos, envio de notificações, geracao de relatórios e tratamento de eventos externos. A fila funciona como um amortecedor entre a taxa de produção e a capacidade de consumo, evitando que o produtor precise conhecer detalhes da infraestrutura de processamento.

Apesar disso, filas não eliminam a complexidade dos sistemas distribuídos. Elas apenas deslocam parte dela para o middleware e para as decisões de projeto. A presença de um broker introduz latência, consumo de armazenamento, necessidade de monitoramento e possíveis pontos de falha. Implementações centralizadas podem se tornar gargalos se não houver replicação, particionamento ou políticas adequadas de alta disponibilidade. Além disso, mensagens antigas, consumidores lentos e filas sem limite podem gerar acumulação de carga e degradar todo o sistema.

Também é necessário projetar idempotência e tratamento de falhas. Mesmo quando a infraestrutura busca entregar uma mensagem apenas uma vez, falhas durante o processamento podem levar a reentregas. Por isso, consumidores devem ser capazes de reconhecer mensagens repetidas ou executar operações de forma segura. Confirmações de consumo, filas de mensagens mortas, limites de tentativa, timeouts, rastreamento e métricas são práticas importantes para transformar o modelo teórico em um sistema robusto.

7 Conclusão

Filas de mensagens são um mecanismo fundamental de comunicação indireta em sistemas distribuídos. Elas oferecem um modelo simples, no qual produtores depositam mensagens e consumidores as retiram, mas esse modelo sustenta propriedades importantes: desacoplamento espacial, desacoplamento temporal, persistência, controle de fluxo, balanceamento de trabalho e integração entre aplicações heterogêneas.

A análise mostra que o valor das filas está menos na estrutura FIFO em si e mais no conjunto de garantias e abstrações fornecidas pelo middleware. IBM MQ, Jakarta Messaging, RabbitMQ e Spring AMQP exemplificam diferentes níveis dessa pilha: produto corporativo, especificação padronizada, broker de mensageria e framework de aplicação. Em todos os casos, a fila atua como intermediária que reduz dependências diretas entre componentes.

Entretanto, o uso de filas exige critérios de projeto. E preciso configurar durabilidade, persistência, confirmações, transações, políticas de reentrega, segurança e observabilidade conforme os requisitos da aplicação. Quando bem projetadas, filas de mensagens tornam sistemas distribuídos mais flexíveis e resilientes. Quando usadas sem controle, podem esconder falhas, acumular atrasos e dificultar diagnosticos. Assim, seu emprego deve combinar a simplicidade do paradigma com uma engenharia cuidadosa das garantias operacionais.